

# MIT 6.S191 – Introduction to Deep Learning

## Lecture 5: Deep Reinforcement Learning

Detailed Study Notes

Personal study notes

Lecture: January 7, 2026

### Contents

|           |                                                                  |           |
|-----------|------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>What is Reinforcement Learning?</b>                           | <b>2</b>  |
| 1.1       | Supervised vs. unsupervised vs. reinforcement learning . . . . . | 2         |
| <b>2</b>  | <b>The Reinforcement Learning Loop</b>                           | <b>3</b>  |
| 2.1       | Agent, environment, action, and action space . . . . .           | 4         |
| 2.2       | State versus reward . . . . .                                    | 4         |
| <b>3</b>  | <b>Return and Discounting</b>                                    | <b>4</b>  |
| <b>4</b>  | <b>The Q-Function</b>                                            | <b>5</b>  |
| 4.1       | From a Q-function to a policy . . . . .                          | 6         |
| <b>5</b>  | <b>Why Q-Values Can Be Counterintuitive: Atari Breakout</b>      | <b>7</b>  |
| <b>6</b>  | <b>Deep Q-Networks (DQN)</b>                                     | <b>8</b>  |
| 6.1       | The Bellman target . . . . .                                     | 9         |
| 6.2       | End-to-end DQN loop . . . . .                                    | 10        |
| <b>7</b>  | <b>Limitations of Q-Learning</b>                                 | <b>11</b> |
| <b>8</b>  | <b>Policy Learning and Policy Gradients</b>                      | <b>12</b> |
| 8.1       | Discrete versus continuous action spaces . . . . .               | 12        |
| <b>9</b>  | <b>Training Policy Gradients: Self-Driving Case Study</b>        | <b>13</b> |
| 9.1       | Policy-gradient loss . . . . .                                   | 14        |
| <b>10</b> | <b>Why Real-World RL Is Hard</b>                                 | <b>15</b> |

|                                               |    |
|-----------------------------------------------|----|
| 11 Application: Reinforcement Learning and Go | 17 |
| 12 Connection to Language Models              | 18 |
| 13 Value Learning vs. Policy Learning         | 19 |
| 14 Lecture 5 Big Picture                      | 19 |
| 15 Quick Review Questions                     | 20 |

# 1 What is Reinforcement Learning?

Reinforcement learning (RL) studies how an **agent learns by interacting with an environment**. Unlike ordinary supervised learning, the training data are not a fixed collection of input-label pairs prepared in advance. The agent actively takes actions, observes what happens next, receives feedback as rewards, and thereby generates new experience while learning. This makes RL especially suited to **dynamic, sequential environments**.

The lecture motivates RL using robotics, game play and strategy, and language modeling. Games are particularly useful research environments because the rules and reward signals are well defined, while still allowing difficult long-horizon decision problems.

## Reinforcement Learning: Robots, Games, the World



Figure 1: Applications highlighted in the lecture: robotics, game play and strategy, and language modeling. (MIT 6.S191 Lecture 5 slide 3).

### 1.1 Supervised vs. unsupervised vs. reinforcement learning

|                  | Supervised learning                  | Unsupervised learning                   | Reinforcement learning                              |
|------------------|--------------------------------------|-----------------------------------------|-----------------------------------------------------|
| <b>Data</b>      | $(x, y)$                             | $x$                                     | interaction experience:<br>states, actions, rewards |
| <b>Main goal</b> | learn $x \rightarrow y$              | learn underlying structure/distribution | choose actions that maximize future reward          |
| <b>Feedback</b>  | explicit target label                | no labels                               | reward from consequences of actions                 |
| <b>Time</b>      | often examples treated independently | often examples treated independently    | inherently sequential                               |

# Classes of Learning Problems

| Supervised Learning                                                                                                                   | Unsupervised Learning                                                                                                                                | Reinforcement Learning                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data:</b> $(x, y)$<br>$x$ is data, $y$ is label                                                                                    | <b>Data:</b> $x$<br>$x$ is data, no labels!                                                                                                          | <b>Data:</b> state-action pairs                                                                                                                                   |
| <b>Goal:</b> Learn function to map<br>$x \rightarrow y$                                                                               | <b>Goal:</b> Learn underlying<br>structure                                                                                                           | <b>Goal:</b> Maximize future rewards<br>over many time steps                                                                                                      |
| <b>Apple example:</b><br><br>This thing is an apple. | <b>Apple example:</b><br><br>This thing is like<br>the other thing. | <b>Apple example:</b><br><br>Eat this thing because it<br>will keep you alive. |

MIT Massachusetts Institute of Technology MIT Introduction to Deep Learning intro.toddeeplearning.com @MITDeepLearning 1/7/26

Figure 2: The lecture contrasts the three learning paradigms and defines RL by sequential state-action interaction and future reward maximization. (MIT 6.S191 Lecture 5 slide 6).

## Key idea

The defining feature of RL is not simply “learning from rewards.” The learner’s actions influence what data it sees next. Learning and data collection are coupled through interaction with the environment.

## 2 The Reinforcement Learning Loop

At time  $t$ , the agent observes a state  $s_t$  and chooses an action  $a_t$ . The environment responds by moving to a new state  $s_{t+1}$  and producing a reward  $r_t$ . The process repeats:

$$s_t \xrightarrow{a_t} \text{environment} \longrightarrow (s_{t+1}, r_t).$$

# Reinforcement Learning (RL): Key Concepts

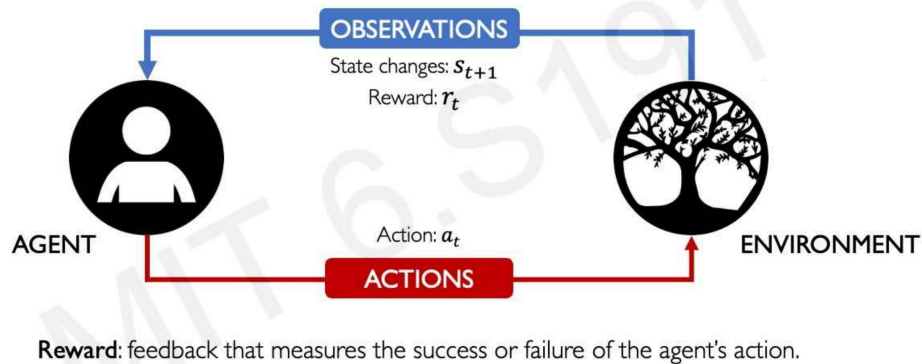


Figure 3: Agent-environment interaction: the agent sends an action and receives the next state and a reward. (MIT 6.S191 Lecture 5 slide 13).

## 2.1 Agent, environment, action, and action space

The **agent** is the decision maker: for example, a delivery drone, a game-playing character, or an autonomous vehicle. The **environment** is the world in which the agent operates. An **action** is a move the agent can execute. The **action space**  $\mathcal{A}$  is the set of allowable actions. In a simple grid world it might be {up, down, left, right}; in driving it could instead contain continuous steering values.

## 2.2 State versus reward

A **state** describes the situation perceived by the agent. In chess, the state can represent the board configuration. A **reward** evaluates the consequence of interaction: it may be positive, negative, or zero. State answers “what situation am I in?” whereas reward answers “how desirable was the outcome?”

Rewards can be **sparse**. A chess agent may receive zero reward throughout the game and only receive a positive or negative signal at the end. This creates the difficult **credit-assignment problem**: actions taken much earlier may have caused a reward that appears much later.

## 3 Return and Discounting

The agent should optimize more than immediate reward. Define the return from time  $t$  as the sum of future rewards. Without discounting,

$$R_t = \sum_{i=t}^{\infty} r_i.$$

The lecture then introduces a discount factor  $0 < \gamma < 1$ :

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

# Reinforcement Learning (RL): Key Concepts

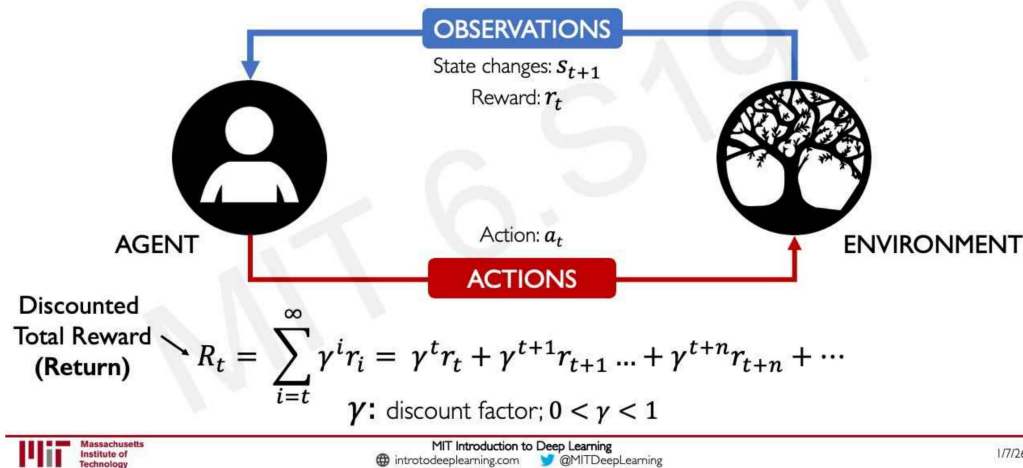


Figure 4: Discounted return: rewards farther into the future receive progressively smaller weights through powers of  $\gamma$ . (MIT 6.S191 Lecture 5 slide 17).

The discount factor expresses how strongly the agent values future rewards. A smaller  $\gamma$  emphasizes near-term outcomes; a value closer to one preserves more influence from distant rewards. The lecture's intuitive analogy is receiving the same amount of money today versus much later: even when the nominal reward is equal, the immediate reward is often preferred.

## Intuition

Discounting does not mean distant rewards are ignored. A reward that arrives only at the end of a long game can still influence earlier decisions, but its contribution to an earlier return is multiplied by the appropriate power of  $\gamma$ .

## 4 The Q-Function

The **Q-function** evaluates a state-action pair. The lecture defines

$$Q(s_t, a_t) = \mathbb{E}[R_t \mid s_t, a_t].$$

It asks: *if I am in state  $s_t$  and take action  $a_t$ , what total future discounted reward should I expect?*

## Defining the Q-function

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Total reward,  $R_t$ , is the discounted sum of all rewards obtained from time  $t$

$$Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t]$$

The Q-function captures the **expected total future reward** an agent in **state,  $s$** , can receive by executing a certain **action,  $a$**

Figure 5: The Q-function connects the current state-action pair to expected total future return. (MIT 6.S191 Lecture 5 slide 18).

A high Q-value means that taking that action in that state is expected to lead to a high return. Importantly, Q-value is not merely the immediate reward. It incorporates the downstream consequences of the action.

### 4.1 From a Q-function to a policy

A **policy**  $\pi(s)$  tells the agent what action to take in a state. If the optimal Q-function were already known, a greedy optimal policy could be obtained by

$$\pi^*(s) = \arg \max_a Q(s, a).$$

## How to take actions given a Q-function?

$$Q(\underset{\substack{\uparrow \\ \text{(state)}}}{s_t}, \underset{\substack{\uparrow \\ \text{(action)}}}{a_t}) = \mathbb{E}[R_t | s_t, a_t]$$

Ultimately, the agent needs a **policy**  $\pi(s)$ , to infer the **best action to take** at its state,  $s$

**Strategy:** the policy should choose an action that maximizes future reward

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q(s, a)$$

Figure 6: Given Q-values, a value-based policy chooses the action with the largest expected future return. (MIT 6.S191 Lecture 5 slide 19).

The lecture divides deep RL into two broad approaches: **value learning**, which learns  $Q(s, a)$  and derives actions from it, and **policy learning**, which learns  $\pi(s)$  directly.

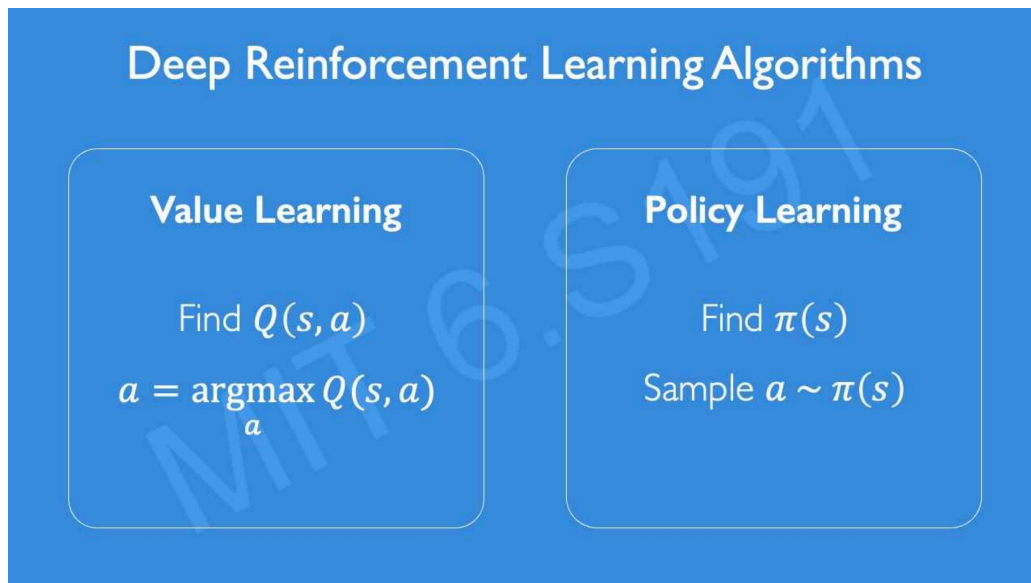


Figure 7: Two approaches emphasized in the lecture: learn a value function and infer a policy, or learn the policy directly. (MIT 6.S191 Lecture 5 slide 20).

## 5 Why Q-Values Can Be Counterintuitive: Atari Breakout

The Atari Breakout example illustrates why humans can have difficulty estimating long-term value. A conservative-looking action may safely return the ball, while a riskier-looking trajectory can eventually break through the side of the bricks and send the ball behind them, producing many future rewards. The

latter state-action pair can therefore have the higher Q-value even if its immediate appearance seems worse.

## Digging deeper into the Q-function

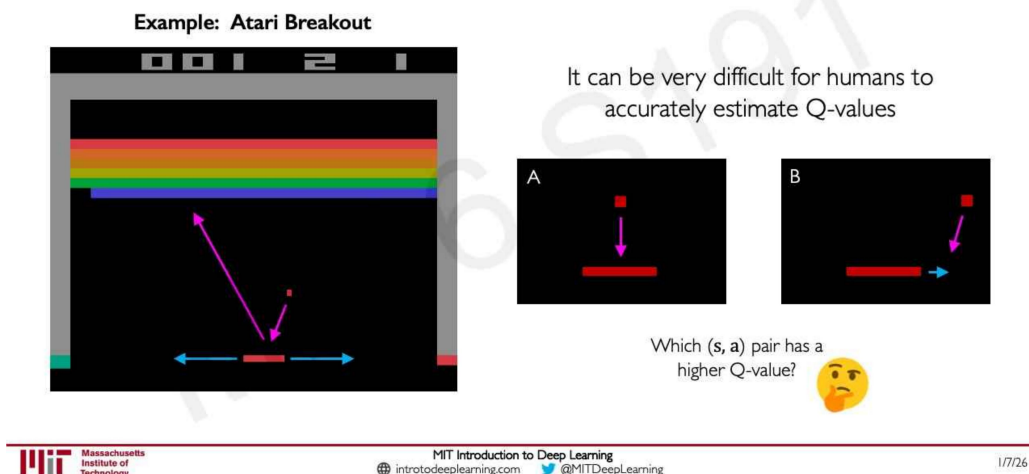


Figure 8: Breakout is used to ask which state-action pair has the greater long-term value. (MIT 6.S191 Lecture 5 slide 22).

### Key idea

Q-learning is fundamentally about **consequences over time**. An action can be valuable because of a chain of states it makes possible, not because it gives the largest immediate reward.

## 6 Deep Q-Networks (DQN)

When states are high dimensional, such as raw game pixels, it is impractical to store a separate Q-value for every possible state-action pair. A **Deep Q-Network** uses a neural network to approximate the Q-function.

A direct formulation would feed both  $s$  and  $a$  into a network and output  $Q(s, a)$ . For a discrete action space, a more efficient formulation feeds the state once and outputs one Q-value for every possible action:

$$f_{\theta}(s) = [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_m)].$$

The action can then be selected by taking the largest output.

# Deep Q Networks (DQN)

How can we use deep neural networks to model Q-functions?

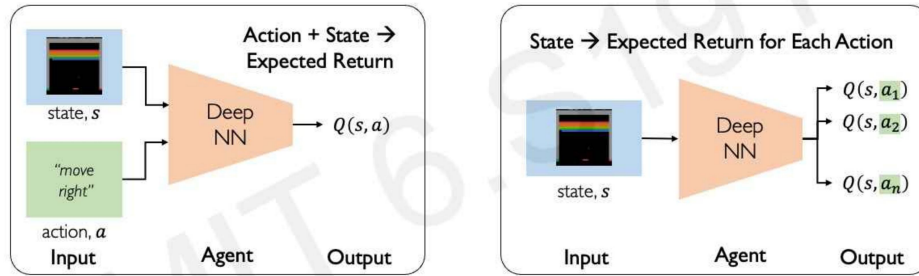


Figure 9: DQN can process a state once and output expected returns for all discrete actions. (MIT 6.S191 Lecture 5 slide 26).

## 6.1 The Bellman target

The training target combines the reward received now with the best predicted future value:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

This is the one-step Bellman target shown in the lecture. The current network prediction is  $Q(s_t, a_t)$ . Training pushes the prediction toward the target.

A squared Q-loss is

$$\mathcal{L}_Q = \mathbb{E} \left[ (y_t - Q(s_t, a_t))^2 \right].$$

Substituting the target,

$$\mathcal{L}_Q = \mathbb{E} \left[ \left( r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)^2 \right].$$

# Deep Q Networks (DQN): Training

How can we use deep neural networks to model Q-functions?

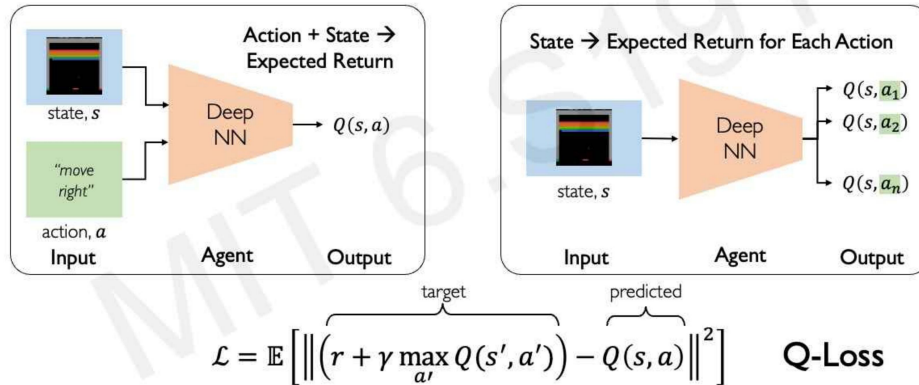


Figure 10: DQN training compares the Bellman target with the network's predicted Q-value. (MIT 6.S191 Lecture 5 slide 30).

## 6.2 End-to-end DQN loop

For a state  $s$ , the network predicts Q-values for all actions. The agent chooses the action with the largest Q-value, executes it in the environment, observes the next state and reward, and uses those transitions to improve the Q-function through backpropagation.

## Deep Q Network Summary

Use NN to learn Q-function and then use to infer the optimal policy,  $\pi(s)$

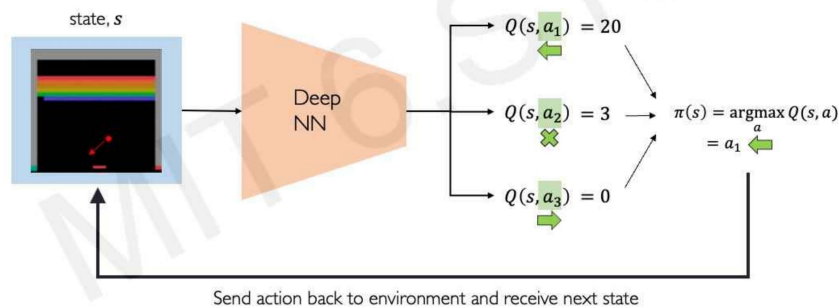


Figure 11: DQN summary: state  $\rightarrow$  neural network  $\rightarrow$  Q-values  $\rightarrow$  argmax action  $\rightarrow$  environment. (MIT 6.S191 Lecture 5 slide 31).

The lecture highlights the landmark Atari work in which convolutional networks learned directly from game pixels and produced Q-values for joystick actions, demonstrating that a relatively general learning algorithm could achieve strong performance across many games.

## DQN Atari Results

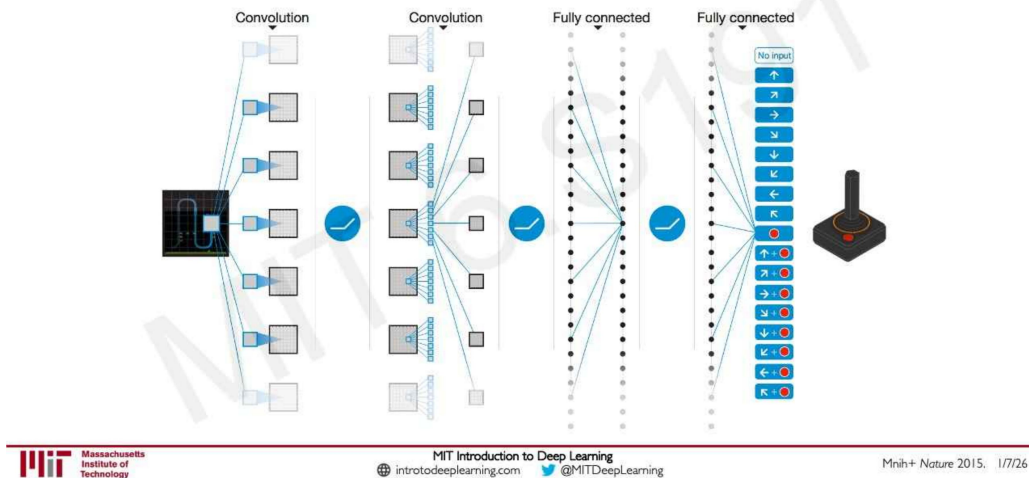


Figure 12: Atari DQN combines visual processing with value prediction over a discrete action set. (MIT 6.S191 Lecture 5 slide 32).

## 7 Limitations of Q-Learning

The lecture emphasizes two limitations of the presented Q-learning formulation. First, enumerating a Q-value for every action is naturally suited to **small discrete action spaces**; it becomes awkward for continuous controls where infinitely many action values are possible. Second, choosing  $\arg \max_a Q(s, a)$  gives a deterministic action rule and does not naturally represent a stochastic policy.

## Downsides of Q-learning

### Complexity:

- Can model scenarios where the action space is discrete and small
- Cannot handle continuous action spaces

### Flexibility:

- Policy is deterministically computed from the Q function by maximizing the reward  $\rightarrow$  cannot learn stochastic policies

To address these, consider a new class of RL training algorithms:  
**Policy gradient methods**

Figure 13: Lecture summary of Q-learning limitations: discrete/small action spaces and deterministic action selection. (MIT 6.S191 Lecture 5 slide 34).

These limitations motivate **policy gradient methods**, which directly parameterize a policy distribution.

## 8 Policy Learning and Policy Gradients

Instead of learning  $Q(s, a)$  and then deriving an action, policy learning directly models

$$\pi_{\theta}(a | s) = P(a | s).$$

For a discrete action space, the network can output probabilities for each action that sum to one. Crucially, the agent **samples** from this distribution rather than always taking its mode.

### Policy Gradient (PG): Key Idea

**DQN:** Approximate Q-function and use to infer the optimal policy,  $\pi(s)$

**Policy Gradient:** Directly optimize the policy  $\pi(s)$

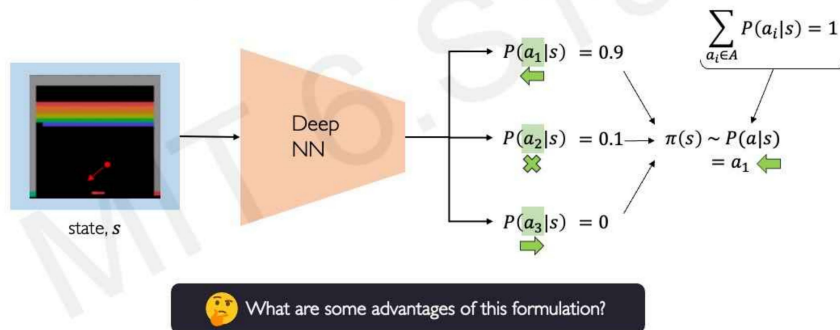


Figure 14: Policy gradient directly predicts an action distribution rather than Q-values. (MIT 6.S191 Lecture 5 slide 37).

This introduces stochasticity. An action with probability 0.9 will usually be chosen, but alternatives can still be sampled. This is different from a purely greedy argmax policy.

### 8.1 Discrete versus continuous action spaces

For discrete actions, the policy may be a categorical distribution over choices such as left, stay, and right. For continuous control, the policy can instead parameterize a probability density. The lecture illustrates a Gaussian policy in which the neural network predicts a mean  $\mu$  and variance  $\sigma^2$ :

$$\pi_{\theta}(a | s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

An action is sampled from that distribution.

# Policy Gradient (PG): Key Idea

**Policy Gradient:** Enables modeling of continuous action space

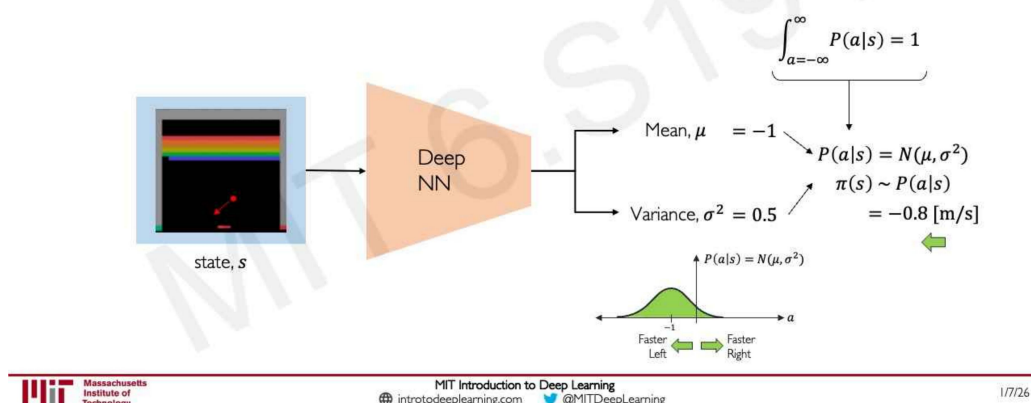


Figure 15: A continuous policy can predict distribution parameters such as  $\mu$  and  $\sigma^2$ , then sample a continuous action. (MIT 6.S191 Lecture 5 slide 40).

The normal distribution is a modeling choice, not a requirement. The policy distribution should be parameterized in a form appropriate to the action space and problem.

## 9 Training Policy Gradients: Self-Driving Case Study

The lecture uses autonomous driving to make the RL components concrete: the **agent** is the vehicle; the **state** comes from sensing such as camera/LiDAR and vehicle position; the **action** is a steering-wheel angle; and a simplified **reward** is distance traveled before a crash.

### Training Policy Gradients: Case Study

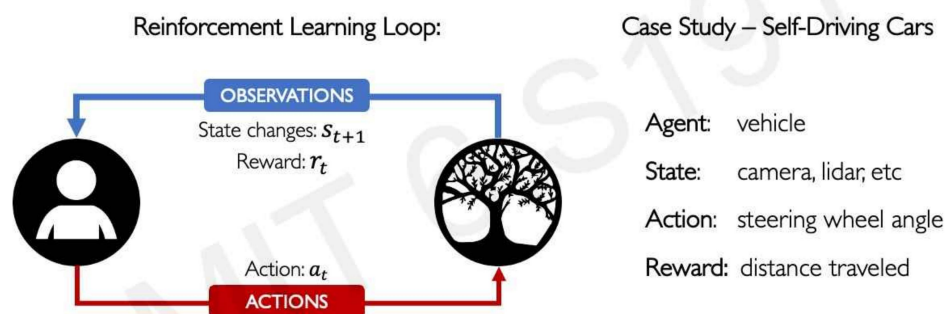


Figure 16: Self-driving case study mapping agent, state, action, and reward into the RL loop. (MIT 6.S191 Lecture 5 slide 41).

The training idea is iterative:

1. Initialize a policy network.
2. Run the policy until termination.
3. Record the states, actions, and rewards along the trajectory.
4. Reduce the probability of actions associated with poor return.
5. Increase the probability of actions associated with high return.
6. Repeat, updating the policy with gradient descent.

## Training Policy Gradients

### Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward

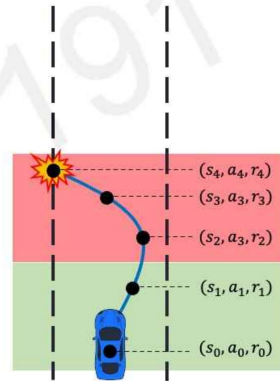


Figure 17: The lecture's policy-gradient training loop: collect a trajectory, then change action probabilities according to obtained rewards. (MIT 6.S191 Lecture 5 slide 42).

### 9.1 Policy-gradient loss

The lecture presents the loss

$$\mathcal{L}_{PG} = -\log \pi_{\theta}(a_t | s_t) R_t.$$

The log-probability says how strongly the current policy supported the sampled action, while  $R_t$  weights that action according to the return that followed it. Gradient descent on this objective increases the probability of actions associated with high positive return and decreases the probability of actions associated with poor return.

# Training Policy Gradients

## Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward

$$\text{loss} = -\overbrace{\log P(a_t | s_t)}^{\text{log-likelihood of action}} \underbrace{R_t}_{\text{reward}}$$

## Gradient descent update:

$$w' = w - \nabla \text{loss}$$
$$w' = w + \underbrace{\nabla \log P(a_t | s_t)}_{\text{Policy gradient!}} R_t$$

Figure 18: Policy-gradient learning weights the log-likelihood of an action by its return. (MIT 6.S191 Lecture 5 slide 46).

The corresponding update can be viewed as

$$\theta \leftarrow \theta + \eta R_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t),$$

where  $\eta$  is the learning rate. This is why the method is called a **policy gradient**: the gradient is taken directly with respect to the policy parameters.

### Intuition

Think of policy gradient as: “Whatever I just did, make it more likely if the eventual return was good and less likely if the eventual return was bad.” The return is what connects a later outcome back to an earlier action.

## 10 Why Real-World RL Is Hard

A major practical problem is that learning often requires executing policies and observing their consequences. In a video game, an agent can fail millions of times safely. A real autonomous vehicle cannot repeatedly crash simply to learn which actions were bad.

# Reinforcement Learning in Real Life

## Training Algorithm

1. Initialize the agent
2. Run a policy until termination
3. Record all states, actions, rewards
4. Decrease probability of actions that resulted in low reward
5. Increase probability of actions that resulted in high reward



Figure 19: The lecture highlights the danger of running an unsafe policy to termination in the real world. (MIT 6.S191 Lecture 5 slide 47).

Simulation therefore plays an important role. The lecture presents **VISTA**, a photorealistic, data-driven simulator for autonomous vehicles, as an example of training policies in simulation before real-world deployment.

## Data-driven Simulation for Autonomous Vehicles

**VISTA**: Photorealistic and high-fidelity simulator for training and testing self-driving cars



Figure 20: VISTA is presented as a data-driven, high-fidelity simulator for training and testing autonomous driving policies. (MIT 6.S191 Lecture 5 slide 48).

## Deploying End-to-End RL for Autonomous Vehicles

**AUTONOMOUS**  
9.7 kph

**EXTERIOR**

**INTERIOR**

2X playback

Policy Gradient RL agent trained entirely within VISTA simulator

End-to-end agent directly deployed into the real-world

**First full-scale autonomous vehicle trained using RL entirely in simulation and deployed in real life!**

MIT Massachusetts Institute of Technology

MIT Introduction to Deep Learning  
introtodeeplearning.com @MITDeepLearning

Amini+ IEEE RA-L 2020. 1/7/26

Figure 21: The lecture shows an RL driving policy trained in simulation and then deployed to a real vehicle. (MIT 6.S191 Lecture 5 slide 49).

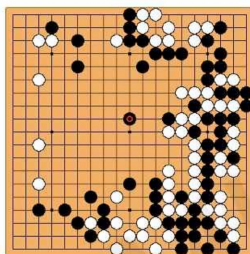
The core challenge is that a simulator must be realistic enough for a policy learned there to transfer successfully to reality.

## 11 Application: Reinforcement Learning and Go

Go is a challenging sequential decision problem because of its enormous number of possible board configurations. The lecture uses AlphaGo as an example of combining multiple learning paradigms rather than relying on RL alone.

### The Game of Go

**Aim:** Get more board territory than your opponent.



| Board Size<br>$n \times n$ | Positions $3^{n^2}$           | % Legal | Legal Positions                 |
|----------------------------|-------------------------------|---------|---------------------------------|
| $1 \times 1$               | 3                             | 33.33%  | 1                               |
| $2 \times 2$               | 81                            | 70.37%  | 57                              |
| $3 \times 3$               | 19,683                        | 64.40%  | 12,675                          |
| $4 \times 4$               | 43,046,721                    | 56.49%  | 24,318,165                      |
| $5 \times 5$               | 847,288,609,443               | 48.90%  | 414,295,148,741                 |
| $9 \times 9$               | $4.434264882 \times 10^{38}$  | 23.44%  | $1.03919148791 \times 10^{38}$  |
| $13 \times 13$             | $4.300233593 \times 10^{80}$  | 8.66%   | $3.72497923077 \times 10^{79}$  |
| $19 \times 19$             | $1.740896506 \times 10^{172}$ | 1.20%   | $2.08168199382 \times 10^{170}$ |

Greater number of legal board positions than atoms in the universe.

Figure 22: Go has an enormous combinatorial state space, making exhaustive search infeasible. (MIT 6.S191 Lecture 5 slide 52).

The lecture's pipeline begins with human expert positions and a supervised policy network that learns to imitate expert moves. The policy is then improved through **self-play and reinforcement learning**. Self-play produces new game trajectories and reward signals without requiring a human to label every move. A value network is also trained to estimate how favorable a board state is.

## AlphaGo Beats Top Human Player at Go

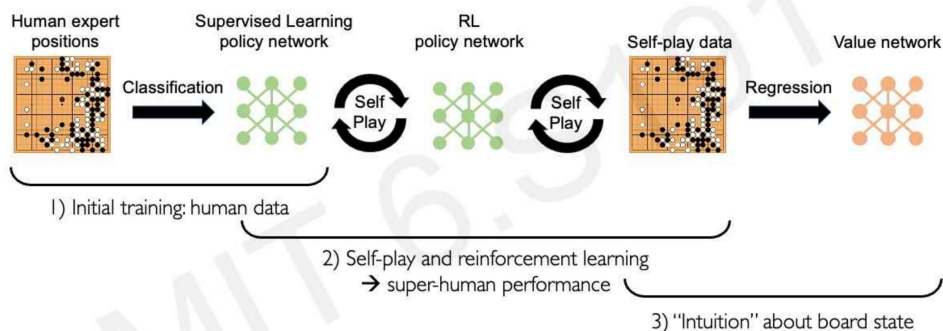


Figure 23: AlphaGo pipeline shown in the lecture: supervised imitation, self-play reinforcement learning, and value learning. (MIT 6.S191 Lecture 5 slide 56).

This illustrates a useful general pattern: supervised learning can bootstrap a reasonable initial policy, after which RL can optimize beyond direct imitation by learning from outcomes.

## 12 Connection to Language Models

The lecture closes by connecting the same principle to language-model alignment. Initial language-model training can be viewed as imitation through next-token prediction over large text corpora. Preference information can then provide a reward signal that encourages outputs humans prefer and discourages outputs they do not prefer. The key conceptual transition is from **imitating observed behavior** to **optimizing behavior according to a reward or preference signal**.

## 13 Value Learning vs. Policy Learning

|                           | Value / Q-learning                         | Policy learning / policy gradient               |
|---------------------------|--------------------------------------------|-------------------------------------------------|
| <b>What is learned?</b>   | $Q(s, a)$                                  | $\pi_\theta(a   s)$                             |
| <b>Action selection</b>   | $\arg \max_a Q(s, a)$                      | sample $a \sim \pi_\theta(a   s)$               |
| <b>Network output</b>     | expected return for actions                | action probabilities or distribution parameters |
| <b>Discrete actions</b>   | natural fit                                | supported                                       |
| <b>Continuous actions</b> | difficult in the lecture's DQN formulation | naturally modeled with continuous distributions |
| <b>Stochastic policy</b>  | not with pure greedy argmax                | yes                                             |
| <b>Training signal</b>    | Bellman target                             | log-probability weighted by return              |

## 14 Lecture 5 Big Picture

### Deep Reinforcement Learning: Summary

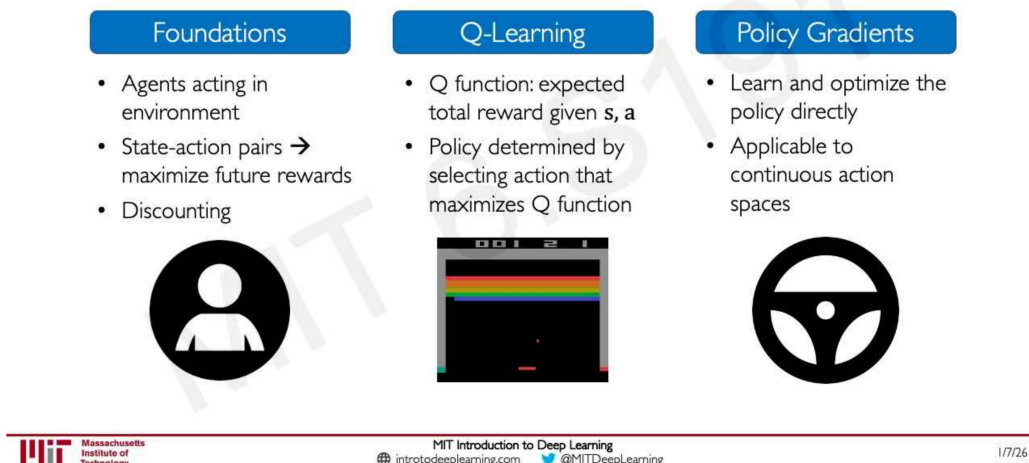


Figure 24: Lecture summary: foundations of agent-environment interaction, Q-learning, and policy gradients. (MIT 6.S191 Lecture 5 slide 57).

The lecture builds deep RL in three layers. First, define the sequential interaction problem through states, actions, rewards, return, and discounting. Second, learn **value**: estimate how good each state-action choice is with a Q-function and use a deep network to approximate it. Third, learn **policy** directly: represent a probability distribution over actions and update it so that rewarding behavior becomes more likely.

- **Agent:** the decision maker.
- **Environment:** the world the agent interacts with.
- **State  $s_t$ :** the agent's current situation/observation.
- **Action  $a_t$ :** what the agent chooses to do.
- **Reward  $r_t$ :** immediate feedback from the environment.
- **Return  $R_t$ :** discounted accumulation of future rewards.
- **Q-function:** expected future return after taking action  $a$  in state  $s$ .
- **Policy  $\pi$ :** rule/distribution for choosing actions from states.
- **DQN:** approximate Q-values with a neural network, then select the largest Q-value.
- **Policy gradient:** directly change policy parameters to make high-return actions more probable.
- **Simulation:** allows risky or repeated RL experience without paying the full cost of real-world failure.

## 15 Quick Review Questions

1. Why is RL described as learning in a dynamic environment?
2. What is the difference between a state and a reward?
3. Why can sparse rewards make learning difficult?
4. What does the discount factor  $\gamma$  control?
5. Why is  $Q(s, a)$  different from the immediate reward  $r_t$ ?
6. If the Q-function were known, how would a greedy policy choose an action?
7. Why does a DQN often output one Q-value per discrete action from a single state input?
8. What is the Bellman target used in DQN training?
9. Why is the presented Q-learning approach awkward for continuous action spaces?
10. How does policy learning differ from value learning?
11. Why do policy gradients sample from  $\pi(a | s)$  instead of always taking an argmax?
12. How can a Gaussian policy represent continuous actions?
13. What does  $-\log \pi(a_t | s_t) R_t$  encourage when  $R_t$  is high?
14. Why is simulation especially important for real-world RL?
15. How did the lecture describe the progression from supervised imitation to reinforcement learning in Go and language-model alignment?

## Source note

These notes are based on the supplied MIT 6.S191 Lecture 5 slides and the accompanying written lecture notes. Slide images are included as study references and are labeled by their original slide number.